

10 aulas

Fundamentos de Programação com Java

Introdução à lógica de programação

Objetivos da Aula

- Compreender o que é programação;
- Identificar problemas computacionais;
- Entender o conceito de algoritmo;
- Representar algoritmos e pseudocódigos e fluxogramas;

O que é programação?

Programar é criar uma sequência de instruções que o computador executa para resolver um problema.

Essas instruções formam um programa.

O computador não pensa

O computador não possui compreensão própria do problema. Ele apenas executa instruções previamente definidas.

Como devem ser as instruções?

Como o computador apenas executa instruções essas instruções precisam ser claras e bem organizadas.

- Lógica: as etapas precisam fazer sentido dentro de uma sequência coerente;
- Ordem: a posição de cada instrução influencia diretamente o resultado;
- Precisão: cada comando deve ser descrito sem ambiguidades.

Programação no cotidiano

Mesmo quando não percebemos, interagimos com programas de computador o tempo todo no nosso dia a dia.

- Aplicativos de celular;
- Jogos digitais;
- Redes sociais;
- Plataformas de streaming;
- Sistemas bancários;

O que todos esses sistemas têm em comum?

Todos funcionam a partir de programas.

Ou seja, conjuntos de instruções que dizem ao computador o que fazer.

Que possuem... Lógica, Ordem e Precisão!

Quantos programas você já usou hoje?

Por que a programação é necessária?

O computador, por si só, não sabe como executar tarefas úteis.

É preciso indicar exatamente o que ele deve fazer.

- (Entrada) - Quais dados devem ser recebidos;
- (Processos) - Quais operações devem ser realizadas;
- (Decisão) - Quais decisões devem ser tomadas;
- (Saída) - Quais resultados devem ser apresentados;

Exemplo (Sistema Bancário)

Para transferir dinheiro entre contas, o Sistema precisa saber:

- Qual conta envia?
- Qual conta recebe?
- Qual valor será transferido?
- A conta de destino tem saldo suficiente?
- Atualizar os saldos?

O papel do programador

O programador transforma um problema do mundo real em uma sequência de instruções que o computador consiga executar.

Programar não começa no código!

Antes de escrever um programa

Antes de começar a programar, é preciso organizar a solução do problema.

- Compreender o problema;
- Identificar os dados envolvidos;
- Organizar a solução em etapas;
- Só então escrever o programa;

Ideia importante!

Programar não é apenas memorizar comandos, Programar é estruturar corretamente o raciocínio.

Programação como resolução de problemas

A essência da programação está na resolução de problemas.

Um problema pode ser simples, como calcular a média de duas notas, ou mais complexo, como controlar um sistema industrial ou processar grandes volumes de dados.

Independente da complexidade, a lógica de solução costuma seguir um padrão.

Como resolver um problema com programação?

- Entender o que precisa ser resolvido;
- Decompor o problema em partes menores;
- Definir uma sequência de passos;
- Representar a solução de forma executável;

Isso é o que chamamos de **pensamento algorítmico**.

Exemplo (Cálculo de média)

Problema:

- Calcular a media de duas notas de um aluno;

Entrada:

- As duas notas do aluno;

Processamento:

- Somar as notas e dividir por dois;

Saída:

- O valor da média;

Como nos comunicamos com o computador?

Humanos se comunicam por meio de idiomas, como português ou inglês.

O computador, por outro lado, opera internamente com sinais elétricos representados em linguagem binária.

Binário significa: 0 ou 1.

Linguagens de programação

Escrever diretamente em binário seria impraticável para a maioria dos problemas.

Por isso utilizamos linguagens de programação, que funcionam como uma forma intermediária entre o raciocínio humano e a execução pela máquina.

Exemplos de linguagens de programação

- Java
- Python
- C
- JavaScript

Cada linguagem possui suas próprias regras de escrita.

Neste curso utilizaremos a linguagem **Java**.

Reflexão

Se o computador apenas executa o que foi escrito,
onde realmente nasce a solução do problema?

Na máquina ou no raciocínio do programador?

O que aprendemos até aqui?

- Programação é criar instruções para resolver problemas
- O computador não compreende o problema ele apenas executa instruções.
- Programar envolve organizar o raciocínio antes de escrever código.
- Linguagens de programação permitem comunicar essas instruções ao computador.

Como o computador executa instruções

Depois de entender o que é programação, o próximo passo é compreender como o computador executa as instruções que recebe.

Isso é importante porque o computador não interpreta intenções: ele apenas executa.

Como as instruções são executadas

De modo geral, o computador executa instruções de forma:

- Sequencial: uma após a outra;
- Literal: exatamente como foram escritas;
- Determinada: seguindo regras precisas de execução;

Ideia central

O computador não distingue, sozinho, uma instrução correta de uma instrução logicamente equivocada. Ele apenas executa.

Execução sequencial

Na forma mais básica, um programa é executado passo a passo, seguindo a ordem em que as instruções aparecem.

O computador executa a primeira instrução, depois a segunda, depois a terceira, e assim por diante.

Exemplo (Cálculo de média)

Considere a tarefa de calcular um média de duas notas. Uma sequência coerente de ações seria:

1. Ler a primeira nota.
2. Ler a segunda nota.
3. Somar as duas notas.
4. Dividir o resultado por 2.
5. Exibir a média.

Importância da ordem das instruções

Considere duas possíveis sequências para calcular a média de duas notas.

Opção 1:

1. Dividir a soma por 2.
2. Somar as duas notas.

Opção 2:

1. Somar as duas notas.
2. Dividir o resultado por 2.

Análise da resposta

A sequência correta é a Opção 2.

Na opção 1, o algoritmo tenta dividir um valor que ainda não foi calculado.

A soma das notas ainda não foi realizada, portanto o valor necessário para na divisão ainda não existe naquele momento.

Análise da resposta

Uma instrução só pode utilizar informações que já tenham sido produzidas nas etapas anteriores.

Quando a ordem das instruções é quebrada, o processo perde a coerência lógica.

Ideia central

Em programação, cada etapa depende dos resultados produzidos anteriormente.

Reflexão

Se uma etapa depende de um resultado anterior, o que acontece quando tentamos executá-la cedo demais?

Execução literal

Além de seguir a ordem das instruções, o computador executa cada comando exatamente como ele foi escrito.

O computador não interpreta intenções. Ele apenas executa instruções.

Se a instrução estiver errada, o computador executará o erro.

Exemplo

Suponha que queremos calcular a soma de dois valores.

Valor1 = 8

Valor2 = 5

Resultado = valor1 – valor2

O computador calculará 3, mesmo que a intenção fosse obter 13.

Ideia central

O computador não sabe qual era a intenção do programador. Ele apenas executa a instrução que foi escrita.

Tipos de erro em programação

Durante o desenvolvimento de programas, dois tipos de erro aparecem com frequência:

- Erro de sintaxe;
- Erro lógico;

Entender essa diferença é fundamental para aprender a programar.

Erro de sintaxe

O erro de sintaxe acontece quando uma instrução é escrita de forma incompatível com as regras da linguagem.

- Esquecer um símbolo obrigatório;
- Escrever uma palavra-chave incorretamente;
- Montar uma expressão em formato inválido;

Nesse caso, o programa normalmente nem consegue ser executado.

Erro lógico

O erro lógico acontece quando a instrução está escrita corretamente, mas a solução está errada.

O programa executa normalmente, mas produz um resultado incorreto.

A ordem errada das instruções é um exemplo clássico de erro lógico.

Ideia central

Erros de sintaxe impedem a escrita correta do programa.

Erros lógicos impedem a obtenção da solução correta.

A responsabilidade do programador

Como o computador executa exatamente o que foi escrito, a responsabilidade pela construção correta da solução é sempre do programador.

Programar não significa apenas “escrever códigos sem erros”.

A responsabilidade do programador

Também significa:

- Pensar na ordem das etapas;
- Verificar se os dados necessários já existem;
- Definir corretamente as operações;
- Conferir se o resultado resolve o problema;

Ideia central

Programar exige organização do raciocínio
antes mesmo da implementação em uma
linguagem como Java.

Antes do código vem o pensamento.

Exemplo guiado de execução

Considere novamente o problema de calcular a média de duas notas.

Nota1 = 8

Nota2 = 6

Como o computador executaria esse processo?

Execução passo a passo

1. Recebe o valor 8 para nota1.
2. Recebe o valor 6 para nota2.
3. Somar os valores: $8 + 6 = 14$.
4. Dividir o resultado por 2: $14/2 = 7$.
5. Exibir a média: 7.

O que esse exemplo mostra?

O processo ocorre em etapas sucessivas.

Cada resultado intermediário serve de base para o próximo passo.

Programas são executados passo a passo, seguindo uma sequência lógica de ações.

Mini atividade

Analise a sequência de ações abaixo.

1. Exibir o valor final
2. Ler dois números
3. Somar os números

Essa sequência esta correta?

Análise

A sequência não está correta.

O algoritmo tenta exibir um valor antes mesmo de ler os dados e realizar o cálculo.

O problema esta na ordem das ações.

Conclusão

O computador executa instruções:

- Em uma ordem definida;
- Exatamente como foram escritas;

Ele não reorganiza passos, não corrige intenções e não avalia sozinho se a solução faz sentido.

Antes de escrever o código, é necessário organizar corretamente a sequência de ações.

O que é um problema computacional?

Nem todo problema do mundo real pode ser resolvido diretamente por um computador.

Para isso o problema precisa ser descrito de forma estruturada.

Quando um problema é descrito de forma que o computador possa manipulá-lo, temos um problema computacional.

Estrutura de um problema computacional

Todo problema computacional possui três elementos principais:

- Entrada;
- Processamento;
- Saídas;

Entrada → Processamento → Saída.

Ideia central

Para que um computador resolva um problema, precisamos primeiro organizar o problema em dados de entrada, regras de processamento e resultado de saída.

Exemplo

Calcular a media de um aluno.

Para que o computador resolva esse problema, precisamos descrevê-lo de forma estruturada.

Modelagem do problema

Entrada:

Duas notas (nota1 e nota2).

Processamento:

$(\text{nota1} + \text{nota2}) / 2$.

Saída:

Valor da média.

Importância da definição das entradas

Um dos erros mais comuns ao resolver um problema computacional é não definir corretamente os dados de entrada.

- Quantas notas existem
- Quais são essas notas
- Como essas notas serão fornecidas

Sem definir as entradas, não é possível definir o processamento.

Importância do processamento

O processamento corresponde às operações aplicadas aos dados de entrada.

- Operações matemáticas
- Comparações
- Tomadas de decisão
- Repetição

Somar as notas e dividir o resultado pelo número de notas

Importância da saída

A saída representa o resultado produzido pelo algoritmo.

- Um valor numérico
- Uma mensagem
- Um relatório
- Uma decisão

Valor da média

Exemplo completo de problema computacional

Determinar se uma pessoa é maior ou menos de idade.

Estrutura do problema

Entrada:

Idade da pessoa.

Processamento:

Verificar se a idade ≥ 18 .

Saída:

Informar se a pessoa é maior ou menor de idade.

A etapa de modelagem

Transformar um problema real em um problema computacional é chamado de modelagem do problema.

- Identificar as entradas;
- Definir as operações;
- Determinar as saídas;

Uma modelagem mal feita leva a algoritmos confusos ou incorretos.

Reflexão

Antes de escrever um algoritmo, pergunte sempre:

- Quais são as entradas?
- O que precisa ser processado?
- Qual é o resultado esperado?

Conclusão

Todo programa existe para resolver um problema.

Antes de escrever um código, é necessário transformar o problema em uma forma estruturada.

Entrada → Processamento → Saída.

Essa estrutura é a base para a construção de um algoritmo.

O que é um algoritmo?

Depois de identificar um problema computacional, precisamos definir como ele será resolvido.

Um algoritmo é:

Uma sequencia finita e ordenada de passos que descreve como resolver um problema

Sequência de passos

Um algoritmo é composto por uma série de etapas executadas em uma determinada ordem.

Cada passo representa uma ação necessária para avançar na solução.

Se a ordem das etapas for alterada, o resultado pode se tornar incorreto.

Finitude

Um algoritmo precisa terminar após um número limitado de etapas.

Uma instrução como:

“Repita o processo para sempre”.

Não constitui um algoritmo válido para resolver um problema.

Clareza das instruções

Cada etapa de um algoritmo precisa ser descrita de forma clara e sem ambiguidade.

Como o computador executa instruções de maneira literal, qualquer ambiguidade pode gerar resultados inesperados.

Ideia importante

Um algoritmo não deve depender de uma interpretação subjetiva.

Cada passo precisa ser suficientemente claro para que qualquer pessoa ou máquina consiga executá-lo.

Algoritmos no dia a dia

Algoritmos não existem apenas em programação.

Eles estão presentes em diversas atividades do nosso cotidiano.

Vamos analisar um exemplo simples.

Exemplo (Escovar os dentes)

1. Pegar a escova de dentes.
2. Colocar a pasta de dente na escova.
3. Escovar os dentes por aproximadamente dois minutos.
4. Enxaguar a boca com água.

O que esse exemplo mostra?

- Existe uma ordem definida de ações.
- Cada etapa é clara.
- O processo possui um início e um fim.

Essas são exatamente as características de um algoritmo.

Reflexão

O que acontece se executarmos essas etapas fora da ordem?

Por exemplo:

Enxaguar a boca antes de escovar os dentes.

Um algoritmo não precisa ser código.

Ele é uma sequência organizada de passos para resolver um problema.

Reflexão

Algoritmo = Raciocínio estruturado

Algoritmos na computação

Na computação, algoritmos descrevem como um problema deve ser resolvido antes da escrita do código.

Primeiro definimos o algoritmo.

Depois implementamos em uma linguagem.

Exemplo (Cálculo da média)

1. Receber a primeira nota.
2. Receber a segunda nota.
3. Somar as duas notas.
4. Dividir o resultado por 2.
5. Exibir a media.

Ideia central

O algoritmo descreve a solução do problema sem depender de uma linguagem de programação.

Por que algoritmos são importantes?

Algoritmos ajudam a organizar o raciocínio antes da implementação.

O que isso permite?

- Compreender melhor o problema;
- Identificar erros na lógica;
- Simplificar a implementação;
- Comunicar a solução para outras pessoas;

Em projetos maiores, algoritmos são discutidos e analisados antes de qualquer código ser escrito.

Conclusão

Um algoritmo representa a lógica da solução de um problema.

A linguagem de programação define como escrever.

O algoritmo define o que deve ser feito.

Ideia central

Antes de programar, é necessário saber resolver o problema.

Características de um bom algoritmo

Um bom algoritmo deve possuir propriedades que garantem execução correta e resultados consistentes.

Propriedades fundamentais

Finitude:

- Deve terminar após um numero limitado de etapas.

Definição precisa:

- cada passo deve ser claro e sem ambiguidades.

Determinismo:

- mesmas entradas produzem o mesmo resultado.

Efetividade:

- cada etapa deve ser executável na prática.

Generalidade:

- deve resolver uma classe de problemas, não apenas um caso específico.

Formas de representação de algoritmos

Depois de definir a lógica da solução, precisamos representar o algoritmo.

Existem diferentes formas de fazer isso.

- Forma descritiva
- Pseudocódigo
- Fluxograma

Forma descritiva

Algoritmo escrito em linguagem natural.

1. Receber a primeira nota
2. Receber a segunda nota
3. Somar as notas
4. Dividir por 2
5. Exibir a média

- ✓ simples e intuitiva
- ✗ pode gerar ambiguidades

Pseudocódigo

Representação intermediária entre linguagem natural e programação.

Início

Leia nota1

Leia nota2

Media $\leftarrow$ (nota1 + nota2) / 2

Escreve media

Fim

- ✓ mais estruturado
- ✓ próximo do código
- ✓ muito usado no ensino

Fluxograma

Representação gráfica do algoritmo.

Oval → início/fim

Retângulo → processamento

Paralelogramo → entrada/saída

Losango → decisão

Conclusão

O mesmo algoritmo pode ser representado de diferentes formas.

A escolha depende:

- Entender a ideia → descritivo
- Estruturar a lógica → pseudocódigo
- Visualizar o fluxo → fluxograma

Fechamento

Antes de programar, é necessário:

- Entender o problema
- Definir a lógica
- Representar o algoritmo

Só depois disso escrevemos o código.

Problema resolvido

Desenvolver um algoritmo que recebe três notas e calcula a media final de um aluno. Se a media for maior ou igual a 7, o aluno está aprovado. Caso contrário, está reprovado.

Identificar Entrada, Processamento e Saída

Entrada

- Nota 1
- Nota 2
- Nota 3

Processamento

- Calcular a média: $(\text{nota1} + \text{nota2} + \text{nota3}) / 3$
- Verificar se a média é maior ou igual a 7

Saída

- Valor da média
- Situação do aluno (aprovado ou reprovado)

Construção do Pseudocódigo

Início

Leia nota1

Leia nota2

Leia nota3

$Media \leftarrow (nota1 + nota2 + nota3) / 3$

Se media ≥ 7 então

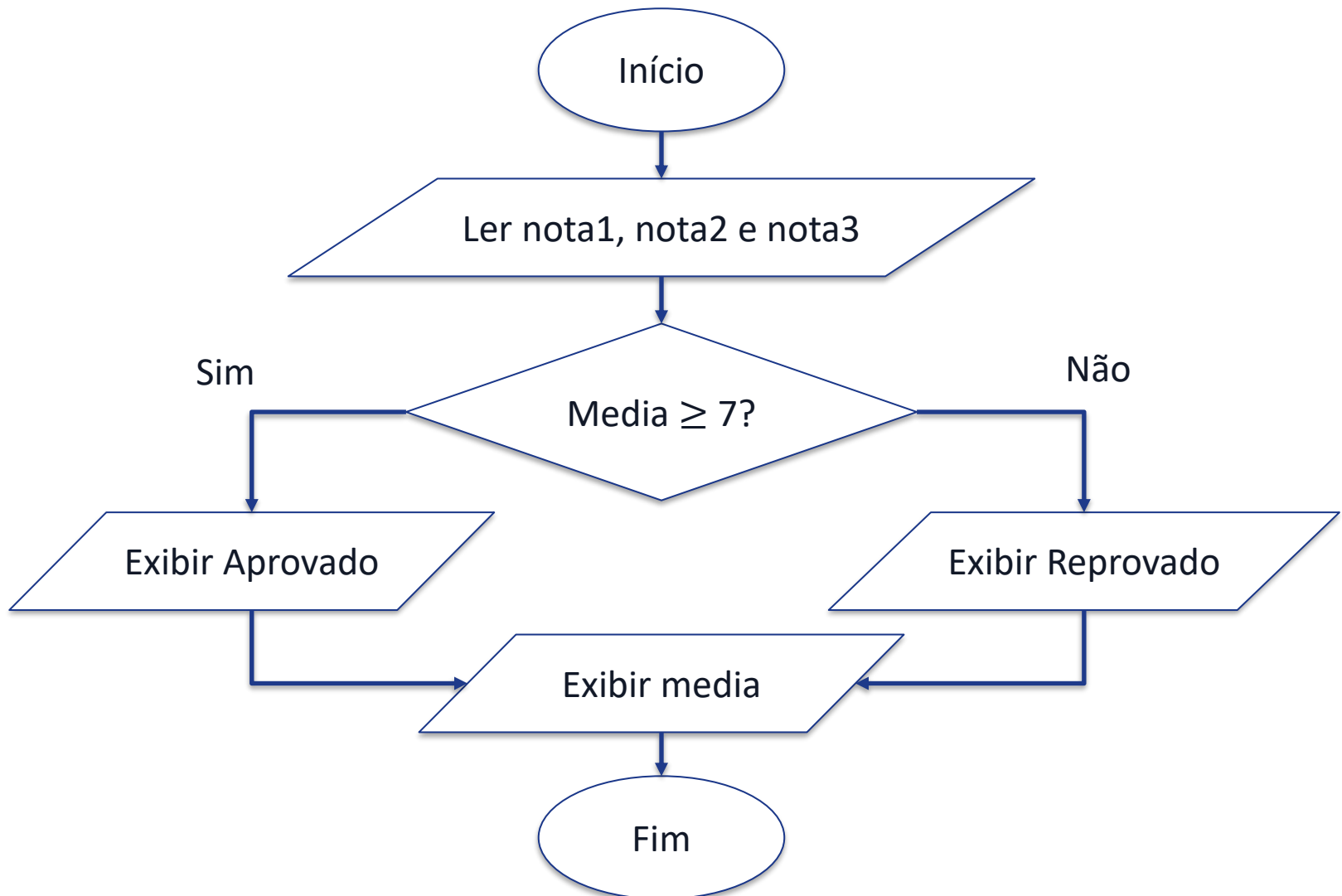
 Escrever “aprovado”

Senão

 Escrever “reprovado”

Fim

Fluxograma



O que aprendemos até aqui?

- O que é programação
- O que é um problema computacional
- O que é um algoritmo
- Características de um bom algoritmo
- Formas de representar algoritmos

Exercícios

1. Calcular o perímetro de um retângulo
2. Calcular a área de um círculo ($\pi = 3,14$)
3. Receber um número e exibir o dobro
4. Verificar maioria
5. Informar se um número é positivo, negativo ou zero

Exercícios

6. Identificar: entrada, processamento e saída (maioridade)
7. Calcular média de três números
8. Calcular área de um triângulo
9. Informar o maior entre dois números

Exercícios

6. Identificar: entrada, processamento e saída (maioridade)
7. Calcular média de três números
8. Calcular área de um triângulo
9. Informar o maior entre dois números

Nos últimos episódios...

O que é um problema computacional?

Um problema computacional é basicamente uma tarefa bem definida que um computador consegue resolver seguindo um passo a passo.

O que é programação?

Programação é escrever esse passo a passo de forma que o computador consiga entender e executar.

Qual papel do programador?

O papel do programador é pensar na solução do problema e transformar isso em um passo a passo claro para o computador executar.

Como devem ser as instruções ao computador?

- Lógicas;
- Ordenadas;
- Precisas;

Como as instruções são executadas?

- Sequencial;
- Literal;
- Determinada;

Qual a estrutura mais simples de um problema computacional?

- Entrada;
- Processamento;
- Saídas;

Tipos de erro em programação

- Erro de sintaxe;
- Erro lógico;

Quais principais características de um algoritmo?

- Finito;
- Sequencial;
- Claro;
- Determinístico;
- Geral;

Quais são as formas de representar um algoritmo?

- Forma descritiva;
- Pseudocódigo;
- Fluxograma;

Instalação e configuração do ambiente

O que é um JDK?

Java Development Kit

O JDK é o ambiente oficial utilizado para criar, compilar e executar programas escritos na linguagem Java.

Ele inclui diversas ferramentas que permitem transformar o código escrito pelo programador em um programa que pode ser executado pelo computador.

Componentes do JDK

- JVM (Java Virtual Machine) – responsável por executar programas Java
- javac – compilador que transforma o código Java em bytecode
- Bibliotecas Java – Conjunto de funcionalidades prontas que podem ser utilizadas pelos programas

JDK é um conjunto de ferramentas

O que é um compilador?

O compilador funciona como um tradutor transformando o código criado em algo que máquina entende

A função principal: Verificar se você cometeu erros de gramática (sintaxe) e gerar um arquivo final pronto para ser executado.

O que é um bytecode?

O **Bytecode** é um meio-termo. Ele não é mais a linguagem humana que você escreveu, mas também ainda não é a linguagem final do processador do seu computador.

Por que ele existe?

Antigamente, se você compilasse um programa no Windows, ele não rodava no Mac.

O que é um bytecode?

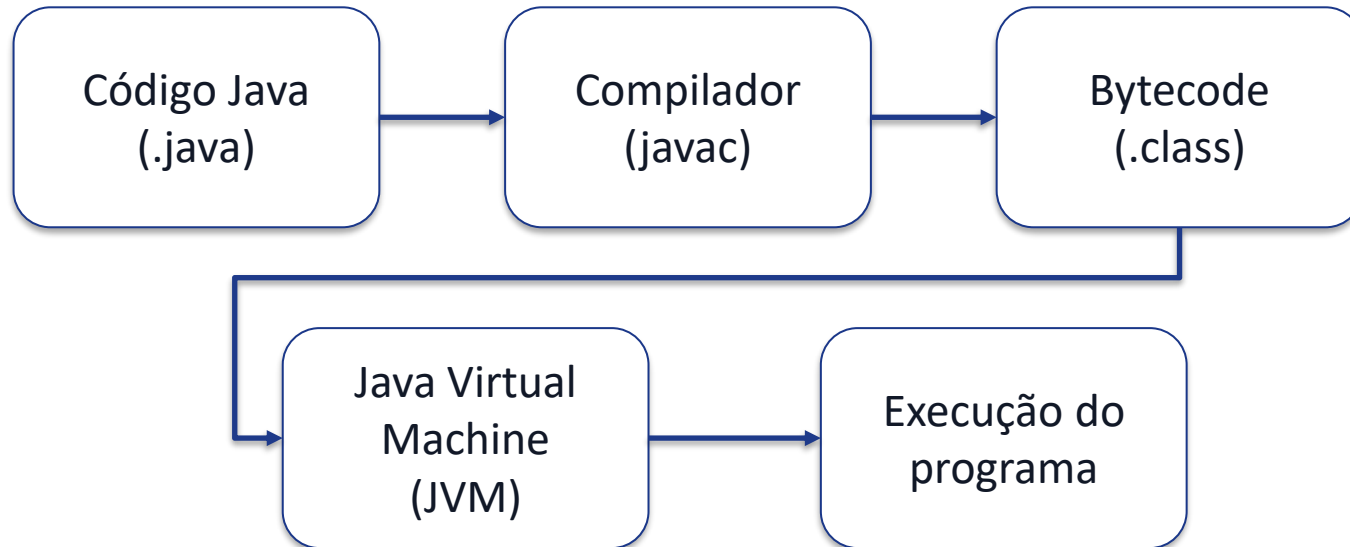
A solução do Java:

O compilador (javac) transforma seu código em **Bytecode**.

Onde ele roda?

O Bytecode é universal. Ele "roda" dentro de uma máquina virtual (como a JVM). É como se fosse uma língua universal que qualquer sistema consegue entender, desde que tenha o "tradutor" (a JVM) instalado.

Esquema de como funciona um programa em Java



Importante

Uma das principais características do Java é que programas compilados podem ser executados em diferentes sistemas operacionais.

Isso ocorre porque o código é executado pela JVM, que funciona como uma camada intermediária entre o programa e o sistema operacional.

Onde podemos baixar o Java?

O Java pode ser encontrado no site:

<https://www.java.com>



Java para executar aplicativos de desktop

Para Usuários Finais em um Desktop ou computador Laptop

[Download de Java para Desktops](#)

[O que é o Java](#)

[Ajuda para usuários finais](#)

Desenvolvedores e Administradores de Empresas

Downloads e recursos gratuitos do JDK (Java Development Kit) da Oracle, a administradora do Java

[Downloads para Desenvolvedores](#)

[Recursos do Desenvolvedor](#)

[Recursos Corporativos](#)

[Acesso Antecipado ao OpenJDK](#)

Onde podemos baixar o Java?

Java 26, Java 25, Java 21, and earlier versions available now

JDK 26 is the latest release of the Java SE Platform.

JDK 25 is the latest *Long-Term Support (LTS)* release of the Java SE Platform.

JDK 21 is the previous Long-Term Support (LTS) release of the Java SE Platform.

Earlier JDK versions are available below.

[Learn about Java SE Subscription](#)

JDK 26 **JDK 25** **JDK 21**

Java SE Development Kit 26 downloads

JDK 26 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 26 will receive updates under these terms, until September 2026, when it will be superseded by JDK 27. Subsequent JDK 26 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will [require a fee](#).

Linux **macOS** **Windows**

Product/file description	File size	Download
x64 Compressed Archive	210.56 MB	https://download.oracle.com/java/26/latest/jdk-26_windows-x64_bin.zip (sha256)
x64 Installer	188.68 MB	https://download.oracle.com/java/26/latest/jdk-26_windows-x64_bin.exe (sha256)
x64 MSI Installer	187.45 MB	https://download.oracle.com/java/26/latest/jdk-26_windows-x64_bin.msi (sha256)

Como saber se minha instalação foi feita com sucesso?

Quando a instalação foi feita com sucesso podemos acessar o terminal e rodar o seguinte comando:

```
Java -version
```

Em caso de sucesso devemos ver algo como:

```
java version "26" 2026-03-17
```

```
Java(TM) SE Runtime Environment (build 26+35-2893)
```

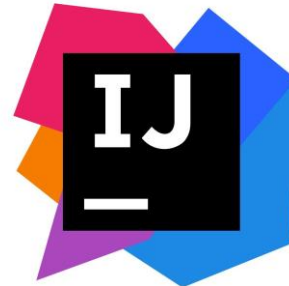
```
Java HotSpot(TM) 64-Bit Server VM (build 26+35-2893, mixed mode, sharing)
```

O que é uma IDE?

Uma **IDE** (Ambiente de Desenvolvimento Integrado) ou um **Editor de Código**.

"Word dos programadores".

Entre eles temos o VS Code e o IntelliJ.



VS Code vs. IntelliJ (VS Code)

Prós:

- Leveza;
- Multilinguagem;
- Ecossistema (Extensões);

Contras:

- Mais manual;
- Menos “Inteligente”;



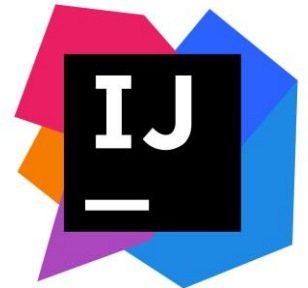
VS Code vs. IntelliJ (IntelliJ)

Prós:

- Feito para Java ;
- Pronto para uso;
- Ferramenta mais utilizada no mercado;

Contras:

- Pesado;
- Alta curva de aprendizado;



Onde podemos baixar essas ferramentas?

VS Code:

<https://code.visualstudio.com/>

IntelliJ:

<https://www.jetbrains.com/idea>



Para VS Code

Baixa a extensão responsável pelo desenvolvimento em Java.



Extension Pack for Java

Microsoft  microsoft.com |  42,446,989 |  (87)

Popular extensions for Java development that provides Java IntelliSense

[Install](#)   Auto Update 

Vamos ao nosso primeiro código em Java?

Antes precisamos entender qual é a sintaxe e como funciona um programa em Java.

Primeiramente, precisamos criar um arquivo .java que receberá o nosso código.

O nome do arquivo sempre tem que levar o mesmo nome da classe que será referencia.

Estruturas básicas de um código em Java

Na linguagem Java, todo código é definido dentro de classes, que constituem a unidade básica de organização e execução de programas.

Em Java, a execução de um programa inicia-se no método main, que define o ponto de entrada da aplicação.

Exemplo

```
public class Main {  
    public static void main(String[] args) {  
        ...  
    }  
}
```

Diagram illustrating the components of the `main` method signature in the provided Java code:

- `public`: Modificador de acesso
- `static`: Modificador static
- `void`: Tipo de retorno
- `main`: método

Modificadores de acesso

Os modificadores de acesso podem variar entre `public`, `private` e `protected`

Membros declarados como `public` podem ser acessados e modificados por qualquer classe, independentemente do pacote.

Membros declarados como `private` podem ser acessados e modificados apenas dentro da classe.

Modificador static

Membros declarados como static pertencem à classe, e não a instâncias específicas.

Membros não estáticos pertencem a instâncias da classe, sendo acessados por meio de objetos.

Como organizar um projeto grande?

Não se faz necessária a criação de uma classe Main para os projetos em Java, no entanto é uma boa prática ter um arquivo .java denominado main que será a raiz dos problemas.

Vamos criar nosso primeiro programa em Java

Vamos criar um programa que tenha como saída uma frase qualquer pré-definida.

Mão a obra!

Variáveis, tipos de dados e operadores

O que é uma variável?

Variáveis são espaços na memória utilizados para armazenar dados durante a execução do programa.

```
int idade;  
double salario;  
String nome;
```

Regra geral:

Tipo + nome + valor (opcional)

Tipos de variáveis (variáveis primitivas)

Tipo	Tamanho	Exemplo	Descrição
byte	8 bits	10	inteiros pequenos
short	16 bits	1000	inteiros médios
int	32 bits	100000	inteiros padrão
long	64 bits	100000L	inteiros grandes
float	32 bits	3.14f	ponto flutuante
double	64 bits	3.14	ponto flutuante (padrão)
char	16 bits	'A'	caractere Unicode
boolean	1 bit	true	lógico

Outros tipos de variáveis

Além dos tipos primitivos, Java possui tipos não primitivos, utilizados para representar estruturas mais complexas.

- String;
- Array;
- Objetos;
- Entradas e saídas do sistema;

Variáveis do tipo String

String é uma classe utilizada para representar sequências de caracteres.

```
String nome = "Bruno";
```

```
nome.length() → tamanho
```

```
nome.toUpperCase() → MAÍSCULO
```

Variáveis do tipo Array

Arrays são estruturas que armazenam múltiplos valores do mesmo tipo.

```
int[] numeros = new int[3];
```

```
int[] numeros = {10, 20, 30};
```

`numeros[0]` → 10

Entrada de dados Scanner

A entrada de dados permite que o usuário forneça informações ao programa.

```
import java.util.Scanner;  
  
Scanner entrada = new Scanner(System.in);  
  
int numero = entrada.nextInt();  
double numero = entrada.nextDouble();  
String texto = entrada.nextLine();
```

Saída de Dados

A saída de dados permite exibir informações ao usuário.

```
System.out.println("Olá");  
System.out.print("Texto");
```

`println()` → quebra a linha

`print()` → não quebra a linha

`\n` é o comando para a quebra de linha manual

**Nos episódios anteriores de
“Programando com Java”...**

O que é um problema computacional?

Um problema computacional é uma tarefa escrita que um computador consegue resolver seguindo um passo a passo.

O que é programação?

Programação é escrever esse passo a passo de forma que o computador consiga entender e executar.

Qual papel do programador?

O papel do programador é pensar na solução do problema e transformar isso em um passo a passo claro para o computador executar.

Como devem ser as instruções ao computador?

- Lógicas;
- Ordenadas;
- Precisas;

Como as instruções são executadas?

- Sequencial;
- Literal;
- Determinada;

Qual a estrutura mais simples de um problema computacional?

- Entrada;
- Processamento;
- Saídas;

Quais os tipos de erro?

- Erro de sintaxe;
- Erro lógico;

Quais principais características de um algoritmo?

- Finito;
- Sequencial;
- Claro;
- Determinístico;
- Geral;

Quais são as formas de representar um algoritmo?

- Forma descritiva;
- Pseudocódigo;
- Fluxograma;

O que é um JDK?

- Java Development Kit – Conjunto de ferramentas para o desenvolvimento em Java

Quais são os principais componentes?

- Java Virtual Machine (JVM)
- javac
- Bibliotecas Java

Como funciona a execução de um programa java?

1. Código (.java)
2. Compilador (javac)
3. Bytecode (.class)
4. JVM
5. Execução

Quais as regras básicas para começar um programa em Java?

- Todo programa esta dentro de uma classe
- O nome do arquivo deve ser igual ao da classe
- Para rodar um arquivo devemos sempre criar um método main

O que são variáveis?

Espaços na memória que armazenam dados

Quais os principais tipos de variáveis?

- int
- double
- boolean
- String

Entrada de dados

- Scanner – Nome do Objeto
- `import java.util.Scanner`
- `Scanner entrada = new Scanner(Scanner.in)`
- `Scanner.nextInt()`
- `Scanner.nextDouble()`
- `Scanner.nextLine()`

Aqui deixa eu corrigir um erro cometido na ultima aula!

Saída de dados

- `System.out.println()`
- `System.out.print()`
- Com `println()` a linha é “quebrada”, com `print()` não
- Para quebrar a linha manualmente usamos `\n`

Exercicios

1. Faça um programa que escreva a frase “Olá mundo!” na tela.
2. Faça um programa que apenas leia uma variável inteira. “Leia”, aqui, refere-se a “ler do teclado”.
3. Faça um programa que escreva o número 4 na tela.
4. Faça um programa que leia e escreva uma variável inteira na tela.
5. Faça um programa que leia uma variável real (*double*).

Exercícios

6. Faça um programa que escreva o número 12.34 na tela.
7. Faça um programa que leia e escreva uma variável real (*double*) na tela.
8. Faça um programa que leia e escreva duas variáveis reais no sentido inverso ao que foram digitadas.

No episódio de hoje...

Operadores aritméticos

- + → soma
- - → subtração
- * → multiplicação
- / → divisão
- % → resto

Concatenação de texto

```
String nome = "Bruno"
```

```
String idade = 31
```

```
String saida = "Olá " + nome + ", parabéns pelos  
seus" + idade + " anos"
```

```
// a saída ter valor igual a
```

```
"Olá Bruno, parabéns pelos seus 31 anos"
```

Cuidados nas operações

Divisões entre inteiros geram resultados inteiros, se faz necessário o cast de uma das variáveis para ter um resultado “com virgula”.

```
int a = 5;
```

```
int b = 2;
```

```
a / b = 2.
```

```
(double) a / b = 2.5;
```

Exercícios

9. Faça um programa que leia duas variáveis reais, realize a soma das variáveis e imprima o resultado na tela.
10. Faça um programa que leia duas variáveis reais, realize a soma das variáveis e imprima as variáveis e o resultado na tela.
11. Refaça o problema anterior, agora dando instruções ao usuário, por exemplo, “Este programa realiza a soma de dois número...”

Exercícios

12. Faça um programa que leia duas variáveis reais, realize a subtração das variáveis e imprima o resultado na tela dando as devidas instruções ao usuário.
13. Faça um programa que leia duas variáveis reais, realize a multiplicação das variáveis e imprima o resultado na tela.
14. Faça um programa que leia duas variáveis inteiras, realize a divisão das variáveis e imprima o resultado na tela.
15. Faça um programa que leia duas variáveis reais, realize a divisão das variáveis e imprima o resultado na tela dando as devidas instruções ao usuário.

Operadores relacionais

- == → igual
- != → diferente
- > → maior que
- < → menor que
- >= → maior ou igual
- <= → menor ou igual

Operadores lógicos

- `&&` → e
- `||` → ou
- `!` → não

Exercícios

1. Faça um programa que leia três números reais, a , b e x , e retorne o valor de y segundo a equação:

$$y=ax+b$$

2. Faça um programa que leia quatro números reais, a , b , c e x , e retorne o valor de y segundo a equação

$$y=ax^2+bx+c$$

Exercícios

3. Construa um algoritmo que calcule a média aritmética de quatro números quaisquer fornecidos pelo usuário.
4. Construa um algoritmo que calcule a média geométrica de três números fornecidos pelo usuário.
5. Construa um algoritmo que calcule a média harmônica de dois números fornecidos pelo usuário.
6. Construa um algoritmo que calcule a área do círculo dado o seu raio.

Exercícios

7. Construa um algoritmo que calcule a área do círculo dado o seu diâmetro.
8. Construa um algoritmo que calcule o volume do cilindro dados a sua altura e o raio.
9. Construa um algoritmo que calcule o volume da esfera dado o seu raio.

Exercícios

10. Dados $a = 3$, $b = 0$, $c = 4$, $d = -1$, $f = 10$, $g = 3.2$, $e = \text{true}$, $h = \text{false}$. Utilize o computador para avaliar as expressões abaixo em:
- i. `!(a + b <= Math.pow(10, 2) || a * c > (double) d / f)`
 - ii. `h && true || (b != 0) || a > d`
 - iii. `(10 % 3.2) > b && !e || (b == 0)`
 - iv. `Math.sqrt(Math.abs(f * g * c / 2 * d)) <= 0`
 - v. `10 % 3 <= 1 && -Math.sqrt(Math.abs(-2)) >= -1`

Estruturas condicionais

Estruturas condicionais

Estruturas condicionais permitem que o programa tome decisões.

Principais estruturas:

- if
- if-else
- switch

Estruturas condicionais (IF)

Executa um bloco de código apenas se a condição for verdadeira.

```
If (condição) {  
    // executa se for verdadeiro  
}
```

Estruturas condicionais (IF-ELSE)

Permite escolher entre dois caminhos:

- Um quando a condição é verdadeira
- Outro quando é falsa

```
if (condição) {  
    // executa se for verdadeiro  
} else {  
    // executado se for falso  
}
```

Múltiplas condições (IF-ELSE IF)

Permite testar varias condições em sequência.

```
if (condição 1) {  
    // executa se condição 1 for verdadeiro  
} else if (condição 2) {  
    // executa se condição 2 for verdadeiro  
} else {  
    // executado se nenhuma for verdadeira  
}
```

Estrutura condicional (SWITCH)

Utilizado para selecionar um entre vários casos possíveis. Funciona melhor com valores exatos (não com intervalos).

```
switch(variavel) {  
    case valor1:  
        // código  
        break;  
    case valor2:  
        // código  
        break;  
  
    default:  
        // caso nenhum seja atendido  
}  

```

Resumo — Estruturas condicionais

As estruturas condicionais permitem controlar o fluxo do programa com base em decisões.

- If: usado para condições simples.
- If-else: usado para duas possibilidades
- If-else if: usado para múltiplas condições
- Switch: usado para comparar um valor com vários casos específicos

Exercícios

1. Construa um programa que leia um número e imprima 0 se ele for positivo e 1 caso contrário.
2. Construa um programa que leia dois números e imprima 0 se eles forem iguais e 2 caso contrário.
3. Construa um programa que leia um número e imprima 1 se ele for divisível por 3 e 0 caso contrário.
4. Construa um programa que leia um número e imprima 0 se ele for positivo e 1 caso contrário.

Exercícios

5. Construa um programa que leia um número x e imprima 0 se ele estiver no intervalo $0 \leq x < 3$ e 1 caso contrário.
6. Faça um programa que leia um número e verifique se ele é raiz da equação $x^3 + 17x^2 + 80x = 0$. Caso seja raiz, o programa deve retornar o valor 1; caso contrário, deve retornar o valor 0.

Exercícios

7. Faça um programa que peça um número e informe se o número é inteiro ou decimal.
8. Faça um programa que peça um valor e mostre na tela se o valor é positivo ou negativo.
9. Faça um programa que peça para o usuário digitar um ano e retorne se ele é bissexto ou não.

Exercícios

10. Faça um programa para a leitura de duas notas parciais de um aluno. O programa deve calcular a média alcançada por aluno e apresentar:
- a) A mensagem "Aprovado", se a média alcançada for maior ou igual a sete;
 - b) A mensagem "Reprovado", se a média for menor do que sete;
 - c) A mensagem "Aprovado com Estrelinha", se a média for igual a dez.

Exercícios

11. O IMC é um critério utilizado para avaliar a condição do peso de um adulto. Ele é calculado pela fórmula:

- $IMC = P / h^2$
- onde: P é o peso (em kg) h é a altura (em metros)
- Com base no valor do IMC, a classificação é feita da seguinte forma:
 - Abaixo de 18,5: Abaixo do peso
 - Entre 18,5 e 25: Peso normal
 - Entre 25 e 30: Acima do peso
 - Acima de 30: Obeso

Exercícios

12. Uma empresa resolve dar um aumento de salário aos seus colaboradores e lhe contrataram para desenvolver o programa que calculará os reajustes.

- a) Faça um programa que recebe o salário de um colaborador e o reajuste segundo o seguinte critério, baseado no salário atual:
 - b) salários até R\$ 280,00 (incluindo) : aumento de 20%
 - c) salários entre R\$ 280,00 e R\$ 700,00 : aumento de 15%
 - d) salários entre R\$ 700,00 e R\$ 1500,00 : aumento de 10%
 - e) salários de R\$ 1500,00 em diante : aumento de 5%
- f) Após o aumento ser realizado, informe na tela:
 - i. o salário antes do reajuste;
 - ii. o percentual de aumento aplicado;
 - iii. o valor do aumento;

Exercícios

13. Dados três comprimentos (p. ex.:a, b e c) estes formam um triângulo se as três condições seguintes forem obedecidas: i) $a < b + c$; ii) $b < a + c$; iii) $c < a + b$; escreva um programa que leia três números positivos e informe se estes números podem ou não formar um triângulo.
14. Dados três comprimentos (p. ex.:a, b e c) e estes formarem um triângulo, imprima como saída se ele é ou não equilátero. Assuma que os números formam um triângulo.
15. Dados três comprimentos (p. ex.:a, b e c) e estes formarem um triângulo, imprima como saída se ele é ou não isósceles. Assuma que os números formam um triângulo.
16. Dados três comprimentos (p. ex.:a, b e c) e estes formarem um triângulo, imprima como saída se ele é ou não escaleno. Assuma que os números formam um triângulo.

Exercícios

17. Escreva um programa que leia três números positivos e classifique o triângulo. Assuma que os números formam um triângulo.
18. Escreva um programa que leia três números positivos e classifique o triângulo. Verifique se os formam um triângulo, e caso não formem, imprima uma mensagem avisando o usuário.
19. Faça um programa que leia três números e mostre o maior deles.
20. Faça um programa que leia dois números e os imprima em ordem decrescente.
21. Faça um programa que leia três números e os imprima em ordem decrescente.

Exercícios

22. Faça um programa que calcule as raízes de uma equação do segundo grau, na forma $a * x^2 + b * x + c = 0$. O programa deverá pedir os valores de a, b e c e fazer as consistências, informando ao usuário nas seguintes situações:
- a) Se o usuário informar o valor de a igual a zero, a equação não é do segundo grau e o programa não deve fazer pedir os demais valores, sendo encerrado;
 - b) Se o delta calculado for negativo, a equação não possui raízes reais. Informe ao usuário e encerre o programa;
 - c) Se o delta calculado for igual a zero a equação possui apenas uma raiz real; informe-a ao usuário;
 - d) Se o delta for positivo, a equação possui duas raiz reais; informe-as ao usuário;

Incrementos

Incremento e Decremento em Java

Incremento e decremento são operações usadas para alterar o valor de uma variável em 1 unidade.

```
int x = 5;
```

```
x++ ou x+=1; // agora x vale 6
```

```
int x = 5;
```

```
x-- ou x-=1; // agora x vale 4
```

Exercícios

1. Imprima a sequência de números 1 a 10 utilizando incremento de variável de controle ($i = i+1$ ou $i++$).
2. Imprima a sequência de números 20 a 15 utilizando decremento de variável de controle ($i = i-1$ ou $i--$).
3. Imprima os pares de 2 a 20 utilizando uma soma própria para a variável de controle.

Estruturas de Repetição

Porque usamos estruturas de repetição?

- Permitem executar um bloco de código várias vezes
- Evitam repetição manual
- Automatizam processos

Então quando eu perguntar para vocês quando devemos utilizar estruturas de repetição vocês devem responder?

- Executar tarefas repetitivas
- Reduzir código
- Automatizar processos

Exemplos

- Mostrar números de 1 até 10
- Ler vários valores
- Exibir menu até que o usuário selecione sair

Quais as principais estruturas de repetição?

Existem três principais estruturas de repetição que podemos usar em um programa em Java:

- While
- For
- Do-while

WHILE

- Em português podemos traduzir para “Enquanto”
- Usando essa definição em português, como você faria uso dessa palavra?

WHILE (Como funciona?)

- Verifica uma condição
- Se for verdadeira → executa
- Repete até a condição ser falsa

```
Int i = 1;
```

```
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}
```

WHILE (Cuidados)

- Looping infinito – Quando a condição nunca se torna falsa

```
Int i = 1;
```

```
while (i <= 5) {  
    System.out.println(i);  
}
```

FOR

- Traduzido para o português significa “por”
- Usando a definição do português, como você usaria essa ferramenta?

FOR

- Usando quando sabemos quantas vezes repetir
- Contém um início, uma condição e um incremento

for (início; condição; incremento)

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

DO-WHILE

- Em português significa “Fazer enquanto”
- Usando a definição do português, como você usaria essa ferramenta?
- De maneira geral funciona de maneira muito parecida com o método WHILE
- Porém garante que o código será rodado ao menos uma vez

DO-WHILE (Como funciona?)

- Executa o bloco
- Depois verifica a condição

```
Int i = 1;
```

```
do {  
    System.out.println(i);  
    i++;  
} while (i <= 5)
```

Comparação

- while → testa antes
- for → controle completo
- do-while → executa ao menos uma vez

Quando usar?

- while → não sabemos quantas repetições
- for → quando sabemos quantas repetições
- do-while → quando precisa executar ao menos uma vez

Quando devemos utilizar estruturas de repetição?

- Executar tarefas repetitivas
- Reduzir código
- Automatizar processos

Exercícios

1. Imprima na tela os numeros de 1 a 100
2. Imprima na tela os números pares de 1 a 100.
3. Imprima na tela os números ímpares de 1 a 100.
4. Faça um programa que imprima na tela os números de 1 a 20, um abaixo do outro.
Depois modifique o programa para que ele mostre os números um ao lado do outro.

Exercícios

5. Imprima na tela os números divisíveis por 4 entre 100 e 1000.
6. Imprima na tela os números de 1 a 1000 que são divisíveis por 13.
7. Faça um programa que leia 5 números e informe o maior número.
8. Imprima a tabuada do número 7 na tela utilizando a estrutura *while*.

Exercícios

9. Imprima a tabuada na tela de um número digitado pelo usuário utilizando a estrutura *while*.
10. Imprima na tela os números de 1 a 1000 que são divisíveis por um número inteiro digitado pelo usuário.
11. Faça um programa que receba dois números inteiros e imprima os números inteiros que estão no intervalo compreendido por eles.

Exercícios

12. Supondo que a população de um país A seja da ordem de 80000 habitantes com uma taxa anual de crescimento de 3% e que a população de B seja 200000 habitantes com uma taxa de crescimento de 1.5%. Faça um programa que calcule e escreva o número de anos necessários para que a população do país A ultrapasse ou iguale a população do país B, mantidas as taxas de crescimento.

Exercícios

13. Imprima na tela os números de x a y que são divisíveis por z , onde x , y e z são dados pelo usuário.
14. Faça um programa que peça dois números, base e expoente, calcule e mostre o primeiro número elevado ao segundo número. Não utilize a função de potência da linguagem.
15. A série de Fibonacci é formada pela sequência 1,1,2,3,5,8,13,21,34,55,... Faça um programa capaz de gerar a série até o n -ésimo termo.
16. Dado um número digitado pelo usuário, verifique se ele é primo.

Comandos de Controle de Fluxo

Ideia central

Durante um laço podemos

- interromper o loop
- pular uma execução

break

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break;  
    }  
    System.out.println(i);  
}
```

continue

```
for (int i = 0; i < 5; i++) {  
    if (i == 2) {  
        continue;  
    }  
    System.out.println(i);  
}
```

Exercícios

1. Desenvolva um programa que percorra uma lista de números inteiros e interrompa a execução assim que encontrar o número 0. Utilize um laço de repetição e o comando adequado para encerrar o loop no momento em que a condição for satisfeita.
2. Crie um programa que percorra os números de 1 a 10 e exiba apenas os números ímpares. Utilize um laço de repetição e o comando apropriado para ignorar os valores que não devem ser exibidos.

Exercícios

3. Um sistema percorre uma lista de nomes e deve parar a busca quando encontrar o nome "Ana". Caso o nome não seja encontrado, o programa deve percorrer toda a lista normalmente. Desenvolva esse programa utilizando um laço de repetição e o comando adequado para interromper a busca assim que o nome for localizado.

Arrays

Porque usamos arrays?

- Imagine o seguinte problema. Temos que manipular 100 notas.
- Da maneira que aprendemos até aqui...

```
int nota1 = 7;
```

```
int nota2 = 8;
```

```
int nota3 = 10;
```

```
...
```

Porque usamos arrays?

- Muitas variáveis faz com que o código fique:
 - “Ruim”
 - Difícil de manter
 - Difícil de analisar
- E se a gente pudesse usar uma estrutura que armazenasse todas essas variáveis para gente?
- Para isso usamos arrays

O que é um array?

- Um array é uma estrutura que, exatamente nos permite armazenar mais de um valor em uma só variável

```
int[] notas = {7, 8, 10}
```

- Todos do mesmo tipo
- Tamanho fixo
- Acesso por índice

Como acessar os dados de um array?

- Para acessar os dados de um array temos que indicar qual posição queremos acessar

```
int[] notas = {7, 8, 10};
```

```
System.out.println(notas[0]); // 7
```

```
System.out.println(notas[1]); // 8
```

```
System.out.println(notas[2]); // 10
```

Importante

- Os índices de um array começam em 0 e vão até $n-1$, onde n é o tamanho dos arrays

```
int[] notas = {7, 8, 10}
```

```
System.out.println(notas[3]);
```

- O que acontece?

Outra maneira de criar um array

- Além da maneira apresentada anteriormente onde:

```
int[] notas = {7, 8, 10}
```

- Podemos fazer da seguinte maneira

```
int[] notas = new int[3];
```

Iniciando arrays manualmente

- Para iniciar arrays de maneira manual temos que indicar os valores de cada posição

```
int[] notas = new int[3];
```

```
notas[0] = 7;
```

```
notas[1] = 8;
```

```
notas[2] = 10;
```

Combinando arrays com estruturas de repetição

- Arrays possuem alta compatibilidade com estruturas de repetição

```
int[] notas = {6, 7, 8}
```

```
for(int i = 0; i < notas.length; i++) {  
    System.out.println(notas[i]);  
}
```

Combinando arrays com estruturas de repetição

- Podemos ainda simplificar essa repetição usando um sistema for-each

```
int[] notas = {6, 7, 8}
```

```
for(int notas: notas) {  
    System.out.println(notas[i]);  
}
```

Exemplo prático

- Vamos fazer um código que faz a soma de 3 notas de um aluno
- Agora vamos permitir que o usuário insira as notas do aluno

O que temos que saber sobre os arrays?

O que é?

- Estrutura para armazenar vários valores

Características:

- Tamanho fixo
- Índice sempre começa em 0

Existe alguma limitação?

- Os arrays possuem tamanho fixo
- Difíceis de alterar
- Pouco flexíveis
- Existe uma estrutura que nos ajuda a contornar isso?

Coleções simples

Porque usamos coleções?

- Estruturas mais flexíveis que arrays
- Tamanho dinâmico
- Mais operações
- Mais usadas no dia-a-dia

Como usar coleções?

- Primeiramente temos que importar a biblioteca ArrayList

```
import java.util.ArrayList;
```

```
ArrayList<Integer> notas = new ArrayList<>();
```

```
ArrayList<Double> notas = new ArrayList<>();
```

```
ArrayList<Boolean> notas = new ArrayList<>();
```

```
ArrayList<String> notas = new ArrayList<>();
```

Como usar coleções?

- Primeiramente temos que importar a biblioteca ArrayList

```
import java.util.ArrayList;
```

```
ArrayList<Integer> notas = new ArrayList<>();
```

```
ArrayList<Double> notas = new ArrayList<>();
```

```
ArrayList<Boolean> options = new ArrayList<>();
```

```
ArrayList<String> nomes = new ArrayList<>();
```

Aqui podemos adicionar elementos

- Diferente dos arrays, nas coleções podemos adicionar valores usando o seguinte comando

```
ArrayList<Integer> notas = new ArrayList<>();
```

```
notas.add(6);
```

```
notas.add(8);
```

```
notas.add(9);
```

Como acessar os valores de uma coleção?

- De maneira similar aos arrays, também temos que fazer o uso de índices para acessar os dados de uma coleção.
- Ao invés de usar `[i]`, usamos `.get(i)`

```
System.out.println(notas.get(0));
```

Outras operações

- Podemos ainda remover elementos da coleção com o método `remove(i)`

```
notas.remove(1);
```

- Para acessar o tamanho de uma coleção usamos o método `.size()`

```
notas.size();
```

Exercícios

1. Crie um array com 5 números inteiros e exiba todos na tela.
2. Dado um array com 5 números, calcule a soma total.
3. Calcule a média dos valores de um array.
4. Encontre o maior número dentro de um array.
5. Encontre o menor número do array.
6. Conte quantos números pares existem no array.

Exercícios

7. Faça um programa que leia vinte inteiros. Utilize para isso um vetor.
8. Faça um programa que leia 20 inteiros e os mostre em tela.
9. Faça um programa que leia n inteiros e os mostre em tela.
10. Faça um programa que leia 30 números reais e faça a média destes números.
11. Faça um programa que leia 20 números reais e faça a média dos 10 últimos.

Exercícios

12. Peça um número ao usuário e verifique se ele existe no array.
13. Crie um novo array com os valores invertidos.
14. Encontre o **segundo maior número** do array.
15. Dado um array, crie outro sem valores repetidos.
16. Crie um programa que armazene 10 notas em um array, calcule a média e exiba a maior e a menor nota.

Mapas

Mapas (Map)

- Estrutura de dados baseada em chave e valor
- Cada chave é única
- Permite acesso rápido aos dados

Estrutura no Java

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
Map<String, Integer> idades = new HashMap<>();
```

Inserindo dados

```
Map<String, Integer> idades = new HashMap<>();
```

```
idades.put("João", 20);
```

```
idades.put("Maria", 25);
```

```
idades.put("Pedro", 30);
```

Acessando dados

```
Map<String, Integer> idades = new HashMap<>();
```

```
int idade = idades.get("Maria");
```

```
System.out.println(idade);
```

Cuidado importante

```
Map<String, Integer> idades = new HashMap<>();
```

```
idades.put("João", 20);  
idades.put("Maria", 25);  
idades.put("Pedro", 30);
```

```
idades.get("Carlos");
```

```
idades.containsKey("Maria")  
idades.containsKey("Carlos")
```

Navegação

- Enquanto arrays e coleções usam índices para acessar os dados
- Nos mapas usamos chaves ou “keys”

Aplicações reais

- Cadastro de usuários
- Sistemas bancários
- APIs (JSON é basicamente Map)
- Configurações de sistema

Exercícios

1. Um professor deseja armazenar o nome de três alunos e suas respectivas idades. Desenvolva um programa que utilize um Map para guardar essas informações e, em seguida, exiba na tela o nome de cada aluno junto com sua idade.
2. Uma aplicação precisa armazenar o nome de algumas cidades e suas populações. Após inserir os dados em um Map, o programa deve exibir a população de uma cidade específica definida no código.

Exercícios

3. Um sistema de loja possui um cadastro de produtos com seus respectivos preços. Desenvolva um programa que armazene alguns produtos em um Map e, posteriormente, atualize o preço de um deles, exibindo o novo valor na tela.
4. Em um sistema simples de cadastro, deseja-se verificar se um determinado nome está presente em um conjunto de dados. Utilize um Map para armazenar alguns nomes e implemente uma verificação que informe se um nome específico existe ou não.

Exercícios

5. Uma escola deseja registrar as notas de alguns alunos. Crie um programa que utilize um Map para armazenar o nome dos alunos e suas respectivas notas, exibindo todas as informações armazenadas.
6. Dado um conjunto de alunos e suas notas armazenados em um Map, desenvolva um programa que solicite o nome de um aluno e exiba sua nota. Caso o aluno não esteja presente no sistema, uma mensagem adequada deve ser exibida.

Exercícios

7. Considere um texto contendo palavras repetidas. Desenvolva um programa que conte quantas vezes cada palavra aparece utilizando um Map. O resultado deve ser exibido no formato palavra → quantidade.
8. Uma pessoa deseja organizar sua lista de compras, armazenando o nome dos produtos e a quantidade desejada. Crie um programa que utilize um Map para representar essa lista e exiba todos os itens com suas respectivas quantidades.

Exercícios

9. Um pequeno sistema de controle de estoque precisa armazenar o nome de produtos e suas quantidades disponíveis. Desenvolva um programa que permita consultar um produto e informe se ele está disponível em estoque.
10. Uma loja deseja identificar qual produto possui o maior preço em seu catálogo. Dado um Map contendo produtos e preços, desenvolva um programa que encontre e exiba o produto mais caro.

Exercícios

11. Uma aplicação precisa calcular o valor total de um conjunto de produtos armazenados em um Map, onde cada produto possui um preço associado. Desenvolva um programa que realize esse cálculo e exiba o resultado final.
12. Um sistema de cadastro permite a remoção de itens. Dado um Map contendo alguns produtos, desenvolva um programa que remova um produto específico e exiba o conteúdo atualizado do mapa.

Exercícios

13. Uma escola deseja classificar seus alunos com base nas notas. Dado um Map contendo nomes e notas, desenvolva um programa que informe, para cada aluno, se ele está aprovado ou reprovado, considerando média mínima igual a 7.
14. Um sistema simples de autenticação precisa validar o acesso de usuários. Utilize um Map para armazenar nomes de usuários e suas respectivas senhas. O programa deve solicitar um login e senha e informar se o acesso foi autorizado ou negado.

Exercícios

15. Considere uma lista de números inteiros que pode conter valores repetidos. Desenvolva um programa que utilize um Map para contar quantas vezes cada número aparece na lista.
16. Uma agenda telefônica precisa armazenar nomes e números de telefone. Desenvolva um programa que permita cadastrar contatos utilizando um Map e possibilite a busca de um telefone a partir do nome.

Exercícios

15. Considere uma lista de números inteiros que pode conter valores repetidos. Desenvolva um programa que utilize um Map para contar quantas vezes cada número aparece na lista.
16. Uma agenda telefônica precisa armazenar nomes e números de telefone. Desenvolva um programa que permita cadastrar contatos utilizando um Map e possibilite a busca de um telefone a partir do nome.

Exercícios

17. Uma loja deseja identificar qual produto foi mais vendido. Dado um Map contendo produtos e suas quantidades vendidas, desenvolva um programa que determine e exiba o produto com maior quantidade.
18. Uma escola deseja separar seus alunos em dois grupos: aprovados e reprovados. Dado um Map contendo nomes e notas, desenvolva um programa que realize essa separação e exiba os dois grupos.

Exercícios

19. Um sistema de mercado precisa calcular o valor total de uma compra. Dado um Map contendo produtos e preços, e uma lista de produtos selecionados, desenvolva um programa que calcule o valor total da compra.
20. Um jogo possui um sistema de pontuação para seus jogadores. Dado um Map contendo nomes e pontuações, desenvolva um programa que identifique e exiba o jogador com maior pontuação e o jogador com menor pontuação.

Set

Ideia central

O que é um Set?

- Estrutura que armazena valores
- Não permite elementos duplicados
- Não possui índices

Criando um Set

```
import java.util.HashSet;  
import java.util.Set;
```

```
Set<String> nomes = new HashSet<>();
```

Adicionando elementos

```
Set<String> nomes = new HashSet<>();
```

```
nomes.add("João");
```

```
nomes.add("Maria");
```

```
nomes.add("João"); // repetido
```

```
[João, Maria]
```

Verificando se existe

```
Set<String> nomes = new HashSet<>();
```

```
nomes.add("João");
```

```
nomes.add("Maria");
```

```
nomes.add("João"); // repetido
```

```
nomes.contains("Maria");
```

Removendo

```
Set<String> nomes = new HashSet<>();
```

```
nomes.add("João");
```

```
nomes.add("Maria");
```

```
nomes.add("João");
```

```
nomes.remove("João");
```

Percorrendo

```
Set<String> nomes = new HashSet<>();
```

```
nomes.add("João");
```

```
nomes.add("Maria");
```

```
nomes.add("João");
```

```
for (String nome : nomes) {  
    System.out.println(nome);  
}
```

Quando usar?

Usamos Set quando:

- Não queremos valores repetidos
- Verificar existência rápida

Exercícios

1. Uma organização está realizando um evento e precisa garantir que cada participante entre apenas uma vez. Para isso, o sistema deve registrar os nomes das pessoas que já acessaram o local. Desenvolva um programa que utilize um Set para armazenar os nomes dos participantes e processe uma sequência de entradas. Para cada nome informado, o sistema deve verificar se a pessoa já entrou anteriormente. Caso seja a primeira vez, o nome deve ser registrado e a entrada permitida. Caso o nome já exista, o sistema deve informar que a entrada já foi registrada anteriormente.

Exercícios

2. Um sistema recebeu uma lista de nomes contendo diversas repetições e precisa gerar uma nova lista contendo apenas valores únicos. Desenvolva um programa que utilize uma estrutura adequada para eliminar os nomes duplicados e apresente o resultado final contendo apenas os nomes sem repetição. Considere que os dados iniciais podem ser fornecidos em uma lista com valores repetidos e que a saída deve conter apenas os elementos distintos.

Funções

Problema

```
int a = 5;  
int b = 7;  
System.out.println((a + b) / 2);
```

```
int c = 8;  
int d = 10;  
System.out.println((c + d) / 2);
```

Ideia

- Funções resolvem repetição de código
- Criar um bloco de código reutilizável
- Executa uma tarefa específica

Estrutura de uma função

```
public static int soma(int a, int b) {  
    return a + b;  
}
```

- public → visibilidade
- static → visibilidade
- int → tipo de retorno
- soma → nome
- (int a, int b) → parâmetros
- return → resultados

Chamando a função

```
int resultado = soma(5, 7);  
System.out.println(resultado);
```

Função vs Código repetido

Sem funções:

- repetição

Com função:

- organizado
- reutilizável

Função sem retorno

```
public static void mensagem() {  
    System.out.println("Olá!");  
}
```

```
mensagem();
```

Parâmetros

```
public static void saudacao(String nome) {  
    System.out.println("Olá " + nome);  
}
```

```
saudacao("Bruno");
```

Exemplo completo

```
public class Main {  
    public static double media(double a, double b) {  
        return (a + b) / 2;  
    }  
  
    public static void main(String[] args) {  
        double resultado = media(8, 6);  
        System.out.println(resultado);  
    }  
}
```

Exercícios

1. Desenvolva um programa que implemente uma função responsável por calcular a soma de dois números inteiros. A função deve receber os dois valores como parâmetros e retornar o resultado da soma. No método principal, chame a função e exiba o resultado.
2. Crie uma função que receba um número inteiro e retorne o seu dobro. No programa principal, utilize essa função com diferentes valores e exiba os resultados obtidos.

Exercícios

3. Desenvolva uma função que receba dois números reais e retorne a média aritmética entre eles. O programa principal deve chamar essa função e imprimir o resultado.
4. Implemente uma função que receba um número inteiro e verifique se ele é par ou ímpar. A função deve retornar uma mensagem indicando o resultado. No método principal, chame a função e exiba a mensagem.

Exercícios

4. Crie uma função que receba o nome de uma pessoa como parâmetro e exiba uma saudação personalizada. A função não deve retornar valor, apenas imprimir a mensagem na tela.
5. Desenvolva uma função que receba três números inteiros e retorne o maior valor entre eles. No programa principal, utilize a função e exiba o resultado encontrado.

Exercícios

5. Implemente uma função que receba a idade de uma pessoa e retorne se ela é maior ou menor de idade. O programa principal deve utilizar essa função e exibir o resultado.
6. Crie um programa que utilize uma função para calcular o valor total de uma compra. A função deve receber o preço de um produto e a quantidade adquirida, retornando o valor total. No método principal, exiba o resultado calculado.

Exercícios

6. Desenvolva uma função que receba um número inteiro positivo e calcule o fatorial desse número. O resultado deve ser retornado pela função e exibido no programa principal.
7. Um sistema precisa calcular a média final de um aluno e informar sua situação. Desenvolva uma função que receba duas notas, calcule a média e retorne se o aluno está aprovado ou reprovado, considerando média mínima igual a 7. O programa principal deve chamar essa função e exibir o resultado.

Complexidade de tempo e espaço

Problema

// Código 1

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}
```

// Código 2

```
for (int i = 0; i < 10000000; i++) {  
    System.out.println(i);  
}
```

Ideia central

Complexidade mede a eficiência

- Quanto tempo demora?
- Quanta memória usa?

Complexidade de tempo

- Mede **quantas operações** o algoritmo faz

Complexidade de espaço

- Mede quanta memória o algoritmo usa

O que é Big-O

O que é o Big-O

- Big-O é uma forma de medir a eficiência de um algoritmo
- Ele descreve como o tempo cresce conforme a entrada aumenta
- Não mede segundos → mede crescimento

Tipos de O

- Constante – $O(1)$

```
int x = 10;
```

- Linear – $O(n)$

```
for (int i = 0; i < n; i++) {  
    System.out.println(i);  
}
```

Tipos de O

- Quadrático – $O(n^2)$

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.println(i + j);  
    }  
}
```

Complexidade de espaço

- Constante – $O(1)$

```
int x = 0;
```

- Linear – $O(n)$

```
int[] vetor = new int[n];
```

- Quadrático – $O(n^2)$

```
int[][] matriz = new int[n][n];
```

Exercícios

1. Considere um programa que realiza apenas uma operação simples, como atribuir um valor a uma variável. Independentemente do tamanho da entrada, o número de operações realizadas permanece o mesmo. Analise esse comportamento e indique qual é a complexidade de tempo desse algoritmo, justificando sua resposta.

Exercícios

2. Um programa percorre todos os elementos de uma lista de tamanho n , executando uma operação para cada elemento. Analise esse comportamento e determine a complexidade de tempo do algoritmo. Explique como o número de operações cresce à medida que o tamanho da entrada aumenta.

Exercícios

3. Considere um algoritmo que utiliza dois laços de repetição aninhados, ambos variando de 0 até n . Para cada iteração do primeiro laço, o segundo laço é executado completamente. Analise esse comportamento e determine a complexidade de tempo do algoritmo, explicando o motivo.

Exercícios

4. Um programa cria um vetor de tamanho n para armazenar dados e, em seguida, cria uma matriz de tamanho $n \times n$. Analise o uso de memória desse programa e determine a complexidade de espaço total. Justifique sua resposta considerando o crescimento da quantidade de memória utilizada.

Exercícios

5. Dois algoritmos são utilizados para resolver o mesmo problema. O primeiro percorre uma lista uma única vez, enquanto o segundo utiliza dois laços aninhados para realizar a mesma tarefa. Considerando entradas de grande tamanho, analise qual dos algoritmos tende a ser mais eficiente. Explique sua resposta com base no crescimento do número de operações.

Trabalho Prático – Sistema de caixa